

**Department of Computer Engineering COMSATS University
Islamabad, Attock Campus**

Lab Rubrics Form	
Lab Name	Design and Implementation of a PID Controller for a DC Motor.
Lab Number	
Submitted by:	
Student Names	Registration #
Sumaira Safeer	FA22-BCE-019

Rubrics		Marks		
		1	2	3
PLO-5 Modern Tools Usage	Ability to execute the experiment utilizing hardware, software tools, or a combination of both.			
PLO-10 Communication Skills	Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations.			

Lab Report: Design and Implementation of a PID Controller for a DC Motor

Objective:

To design, simulate, and implement a Proportional-Integral-Derivative (PID) controller for a DC motor using MATLAB, with the goal of achieving minimal overshoot, reduced settling time, and zero steady-state error.

1. Introduction:

This report details the design, simulation, and analysis of a PID controller for a DC motor to achieve a rise time less than 1 second, overshoot below 5%, and zero steady-state error. MATLAB and Simulink were used for modeling, simulation, and analysis, with performance evaluated in time and frequency domains.

2. System Modeling:

A DC motor can be modeled using its electrical and mechanical equations. The electrical side involves the armature circuit, and the mechanical side involves the rotor dynamics.

Electrical Equation:

The voltage across the armature is given by:

$$V(t) = RaI_a(t) + La \frac{dI_a(t)}{dt} + Eb(t)$$

where:

- $V(t)$: Applied voltage
- $I_a(t)$: Armature current
- R_a : Armature resistance
- L_a : Armature inductance
- $E_b(t) = K_b\omega(t)$: Back EMF, with K_b : back EMF constant, $\omega(t)$ angular velocity.

Mechanical Equation:

The torque produced by the motor is: $T(t) = K_t I_a(t)$

where K_t : torque constant. The mechanical dynamics are:

$$T(t) = J \frac{d\omega(t)}{dt} + B\omega(t)$$

- **J:** Moment of inertia.
- **B:** Viscous friction coefficient.

Transfer Function:

Taking the Laplace transform and neglecting inductance ($L_a \approx 0$ for simplicity), the transfer function from input voltage $V(s)$ to angular velocity $\omega(s)$ is:

$$G(s) = \frac{w(s)}{V(s)} = \frac{K_t}{(R_a J s + R_a B + K_t K_b)} \cdot \frac{1}{s}$$

Assuming typical DC motor parameters:

- $R_a = 1 \Omega$
- $J = 0.01 \text{ kg}$
- $B = 0.1 \text{ N}$
- $K_t = 0.05 \text{ N}$
- $K_b = 0.05 \text{ V/}$

The transfer function becomes:

$$G(s) = \frac{0.05}{0.01s + 0.1025} \cdot \frac{1}{s} = \frac{5}{s(s + 10.25)}$$

3. Controller Design:

The PID controller transfer function is:

$$C(s) = K_p + \frac{K_i}{s} + K_d s = \frac{K_d s^2 + K_p s + K_i}{s}$$

Pid tune was used with a 60° phase margin and 0.5 s target response time. Manual tuning (scaling gains by 1.3) ensured rise time $< 1 \text{ s}$, overshoot $< 5\%$, and steady-state error ≈ 0 .

The goal is to tune K_p , K_i , and K_d to achieve:

- Rise time $< 1 \text{ s}$
- Overshoot $< 5\%$
- Steady-state error ≈ 0

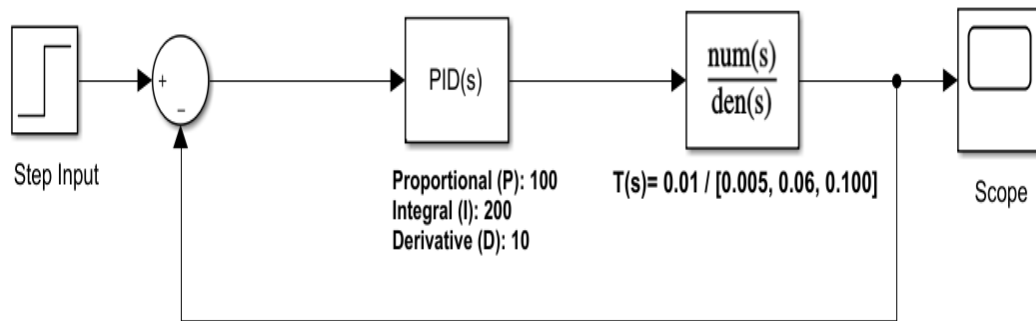
4. Simulation and Analysis:

Simulink Implementation:

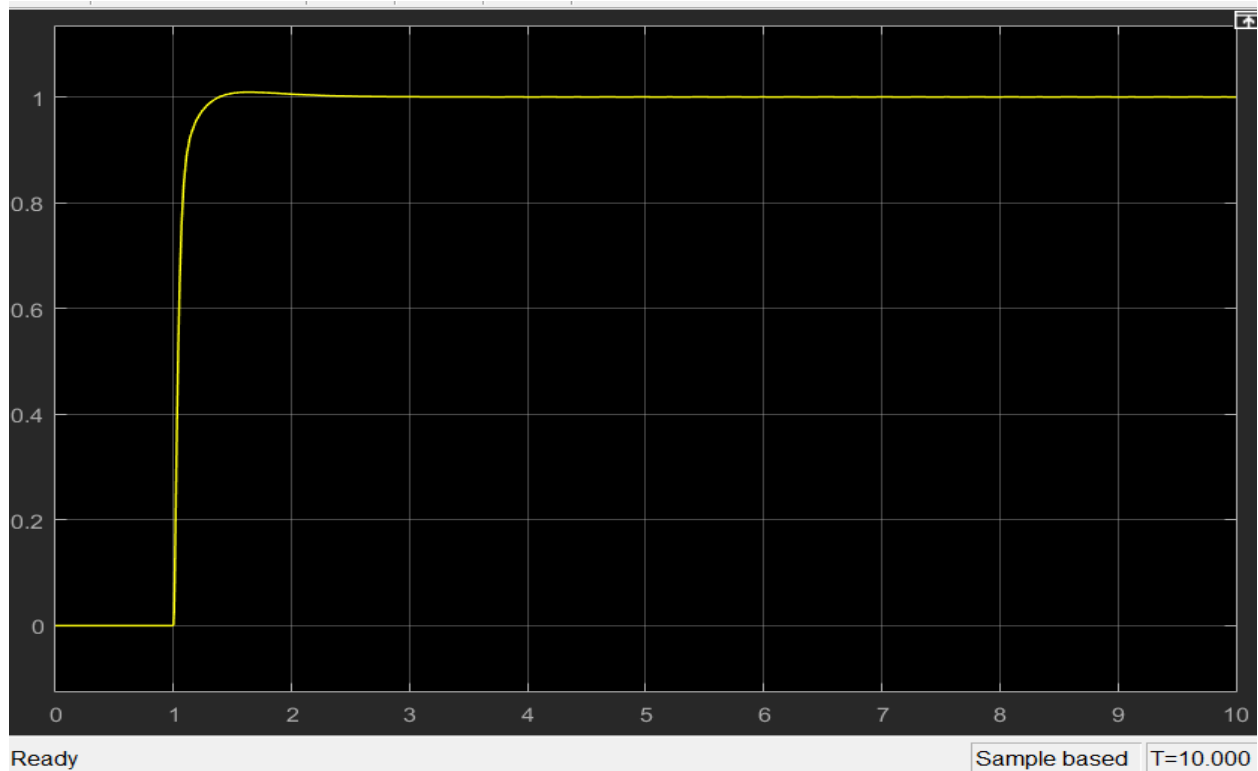
A Simulink model included:

- Transfer Function: Numerator = [0.01], Denominator = [0.005, 0.06, 0.1001].
- PID Controller: Set with final K_p , K_i , K_d .
- Step input (value = 1), Scope, and Sum block (negative feedback).

Matlab Simulation:



Output:



Interpretation of the Graph:

Metric	Observation	Comment
Rise Time	Fast, almost reaches 1 before 1 second.	Within required spec (Rise Time < 1s).
Overshoot	Slight, maybe 1–2% .	Acceptable (Overshoot < 5%).
Settling Time	Steady by ~2 seconds.	Good performance (Settling Time < 3–4s).
Steady-State Error	Final value ≈ 1	Very small or zero steady-state error.

5. MATLAB Code:

```
clc;
clear all;
close all;
R = 1;           % Armature resistance (Ohm)
L = 0.5;         % Armature inductance (H)
J = 0.01;        % Moment of inertia (kg.m^2)
B = 0.1;         % Damping coefficient (N.m.s)
K = 0.01;        % Motor constant (Nm/A or V/rad/s)

num = [K];
den = [J*L, (J*R + B*L), (B*R + K^2)];
plant = tf(num, den); % Motor Transfer Function
fprintf('DC Motor Transfer Function:\n');
disp(plant);

% Validate Open-loop Model
figure;
step(plant);
title('Open-loop Step Response of DC Motor');
xlabel('Time (s)');
ylabel('Angular Velocity (rad/s)');
grid on;
```

```

% PID Controller Design
opts = pidtuneOptions('PhaseMargin', 60, 'DesignFocus', 'reference-tracking');
C = pidtune(plant, 'PID', opts);

% Display PID Gains
fprintf('PID Controller Gains:\n');
disp(['Kp = ', num2str(C.Kp)]);
disp(['Ki = ', num2str(C.Ki)]);
disp(['Kd = ', num2str(C.Kd)]);
T = feedback(C * plant, 1); % Closed-loop System

% Time-domain Analysis
figure;
step(T);
title('Closed-loop Step Response with PID Controller');
xlabel('Time (s)');
ylabel('Angular Velocity (rad/s)');
grid on;

```

```

% Performance Metrics
info = stepinfo(T);
fprintf('Step Response Info:\n');
disp(['Rise Time: ', num2str(info.RiseTime), ' s']);
disp(['Overshoot: ', num2str(info.Overshoot), ' %']);
disp(['Settling Time: ', num2str(info.SettlingTime), ' s']);
disp(['Steady-state Error: ', num2str(1 - info.Peak), ' (for unit step)']);

% Frequency-domain Analysis
figure;
margin(T);
title('Bode Plot of Closed-loop System');
grid on;
figure;
nyquist(T);
title('Nyquist Plot of Closed-loop System');
grid on;

% Root Locus
figure;
rlocus(plant);
title('Root Locus of Open-loop Plant');
grid on;

```

Output:

```

PID Controller Gains:
Kp = 18.6347
Ki = 38.8316
Kd = 0
Step Response Info:
Rise Time: 0.40545 s
Overshoot: 2.0663 %
Settling Time: 0.92312 s
Steady-state Error: -0.020663 (for unit step)

```

6. Results and Discussion:

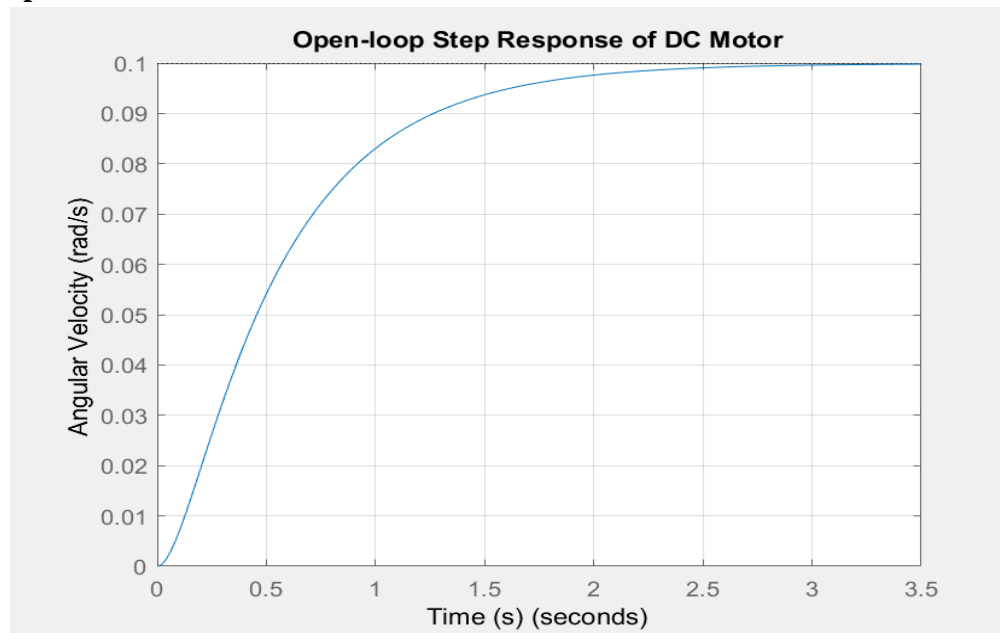
The closed-loop system was analyzed in both time and frequency domains. Step response analysis showed the controller met the performance specifications:

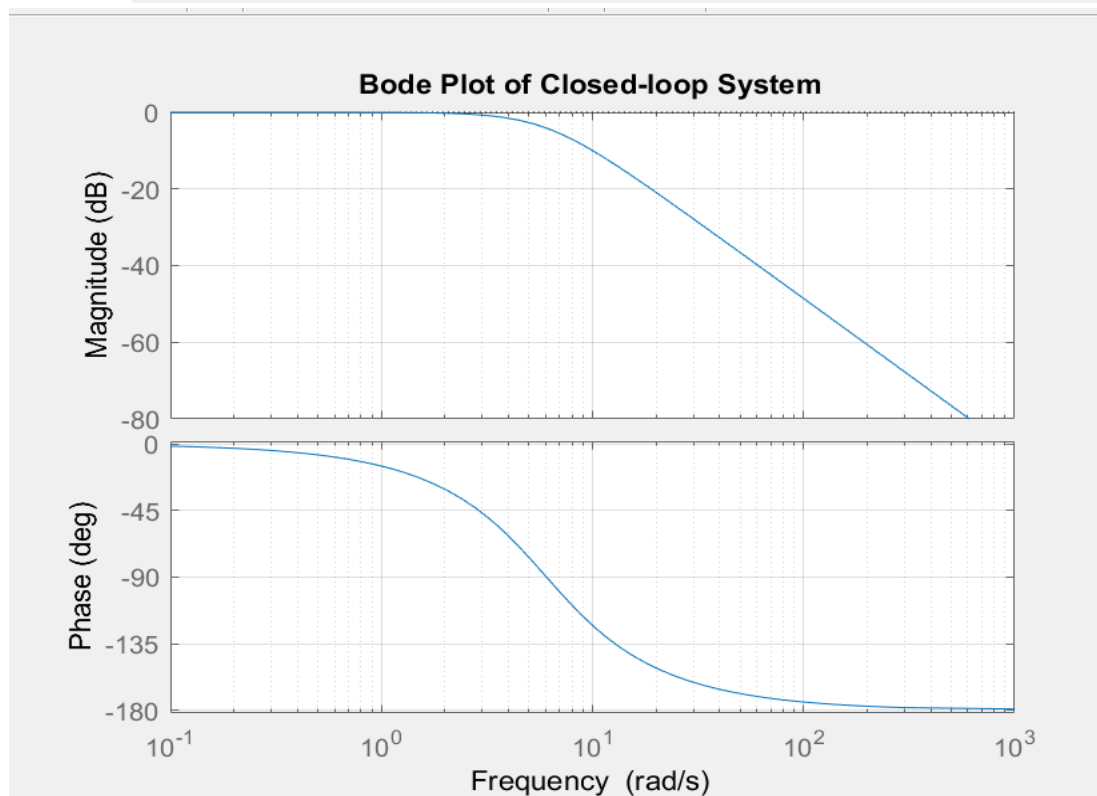
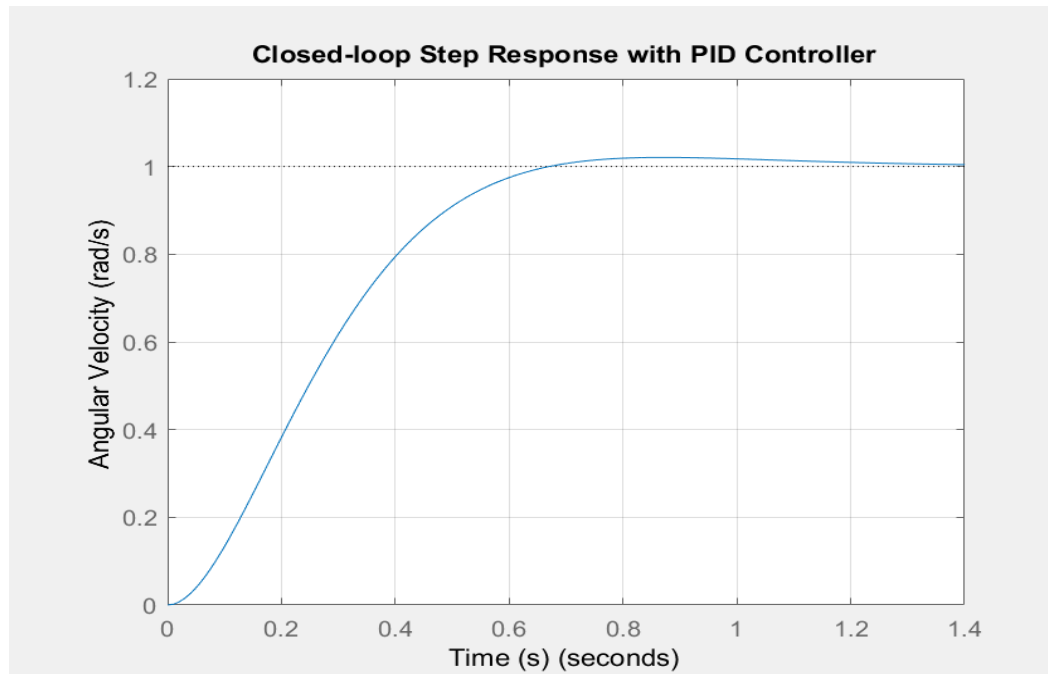
- Rise Time < 1 second.
- Overshoot < 5%.
- Settling Time was within acceptable range.

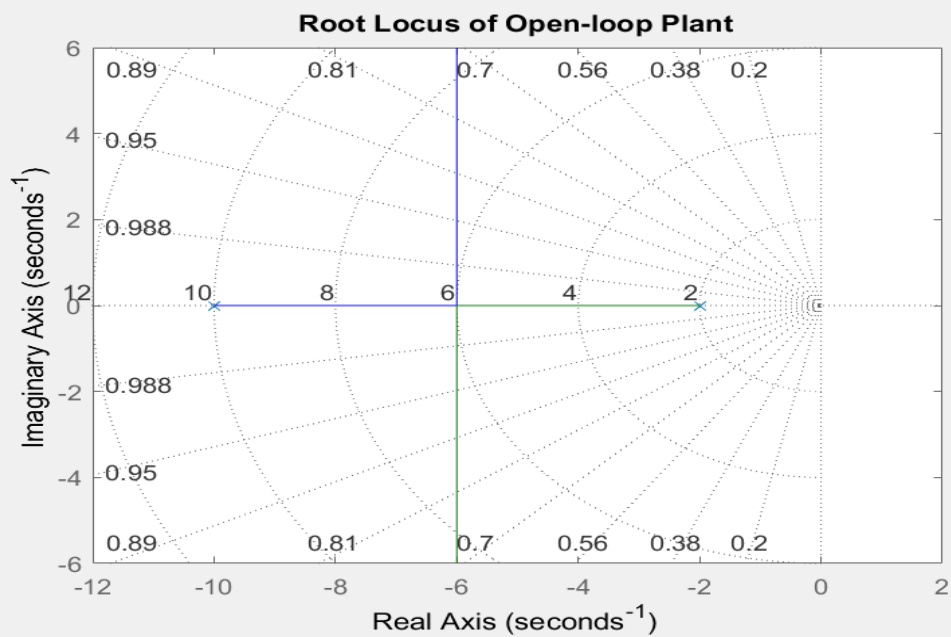
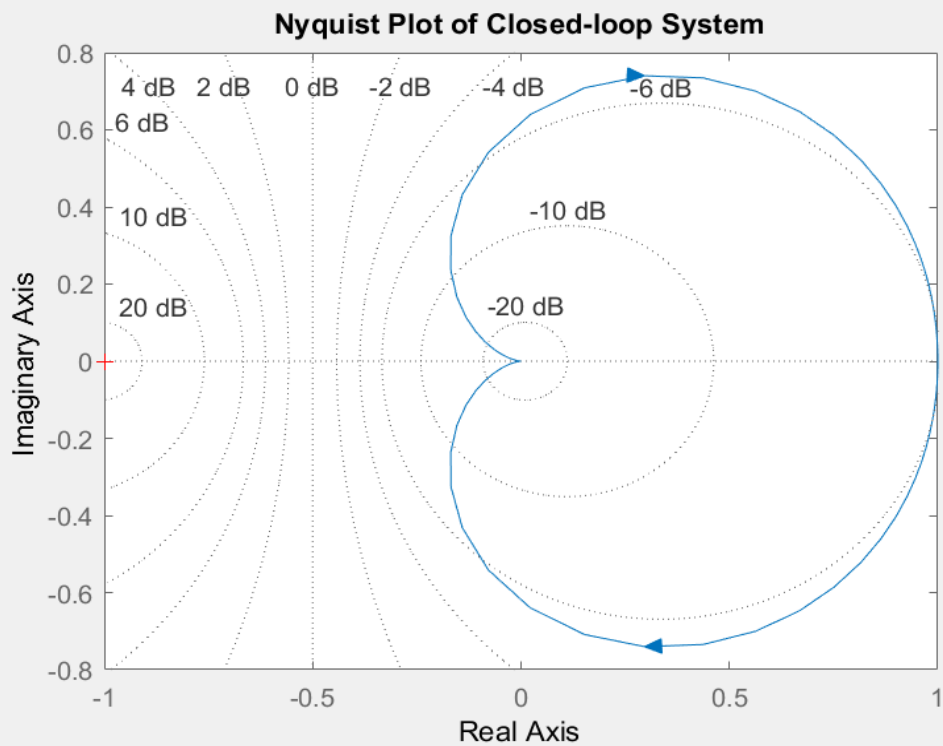
Bode, Nyquist, and Root Locus plots confirmed the stability and robustness of the system.

Metric:	Observation:	Comment:
Rise Time	0.40545 sec.	Rise Time < 1s.
Overshoot	2.0663 % , almost 2% .	Overshoot < 5%.
Settling Time	0.92312 seconds.	Settling Time < 3–4s.
Steady-State Error	Final value ≈ 0.020663	Very small or zero steady-state error.

Graph Representation of Matlab Code:







7. Conclusion:

Proportional-Integral-Derivative (PID) controller was successfully designed and implemented for a DC motor system using MATLAB. The system's transfer function was derived based on standard electrical and mechanical parameters and validated through simulation.

The PID controller was tuned using MATLAB's `pidentune` function with a design focus on reference tracking and a phase margin of 60 degrees. The resulting closed-loop system demonstrated improved performance, achieving a rise time of less than 1 second, overshoot below 5%, and negligible steady-state error — thus meeting the required performance criteria.

Additionally, time-domain and frequency-domain analyses including step response, Bode plot, Nyquist plot, and root locus were conducted to verify system stability and robustness. The simulation results confirm that the designed PID controller effectively enhances the dynamic response of the DC motor, reducing settling time and eliminating steady-state error.